chFnirs: A Tutorial Introduction

Unified second-level analysis for fNIRS data

Dale J. Cohen

2026-11-08

Contents

1 Introduction **2**
1.1 What `chFnirs` does . . . 2
1.2 The second-level analysis problem . . . 2
1.3 When to use `chFnirs` . . . 2

2 Installation **3**

3 The data format `chFnirs` expects **3**
3.1 Building a synthetic example dataset . . . 3

4 The categorical pipeline **5**
4.1 Omnibus testing with BH correction . . . 5
4.2 Activation tests . . . 6
4.3 Gated post-hoc contrasts . . . 7
4.3.1 Why gating is enforced . . . 8
4.4 Effect-size heatmap . . . 9

5 Adding a continuous moderator **10**
5.1 Slopes vs. zero . . . 11
5.2 Gated slopes: do groups have different slopes? . . . 12
5.3 Visualizing slopes . . . 12

6 Writing results by chromophore **15**

7 Design principles to know **15**
7.1 `model` is required . . . 15
7.2 `vcov` has a conditional default . . . 15
7.3 Snake-case output schema . . . 16
7.4 Gated functions error on missing gating column . . . 16
7.5 Effect sizes are always computed . . . 16

8 Common pitfalls **16**

9 Summary **17**

10 Acknowledgments **17**

11 References **17**

1 Introduction

1.1 What `chFnirs` does

`chFnirs` is an R package for the **second-level (group-level) statistical analysis** of functional near-infrared spectroscopy (fNIRS) data. A typical fNIRS study has two analysis stages:

1. **First level** — turn raw optical data into a per-subject, per-channel (or per-region) response estimate for each experimental condition. This is usually done in a toolbox such as NIRS Brain AnalyzIR, Homer3, or NIRS-SPM.
2. **Second level** — pool the per-subject estimates across participants, test omnibus effects of your experimental factors, follow up significant effects with post-hocs, correct for the hundreds of channels/regions you tested, and visualize the results.

`chFnirs` handles the second level. It assumes you have already produced a long-format table of per-subject response estimates (one row per subject × area × condition × chromophore), and provides a consistent set of functions that:

- fit an omnibus model (linear or linear mixed) at every brain area,
- apply Benjamini–Hochberg FDR correction across areas,
- run marginal-mean tests of activation (for categorical factors) or slope-vs-zero tests (for continuous moderators),
- run pairwise post-hocs gated on the omnibus,
- produce effect-size / p-value heatmaps and simple-slope plots,
- write per-chromophore CSVs in a reproducible directory layout.

The package is opinionated about two things: a snake-case output schema that the downstream visualization functions rely on, and an area-wise iteration model in which every function fits a separate model for each brain area (channel, ROI, region, etc.). Everything else — which model to fit, which covariance structure to use, which effect to gate on — is under your control.

1.2 The second-level analysis problem

fNIRS data produces a nontrivial statistical workload. A modest experiment might collect:

- ~ 40 subjects,
- 4 chromophores (HbO, HbR, HbDiff, CBSI),
- 16–80 channels or 4–16 region-of-interest aggregates,
- 2–6 experimental conditions.

That gives you hundreds to thousands of cells. At each cell you want to (a) fit an omnibus model with your experimental factors, (b) correct for multiple testing across areas, (c) run post-hocs where the omnibus is significant, and (d) compute an effect size. Doing this inline in an analysis script leads to several hundred lines of nearly-identical `for` loops, hand-rolled BH correction, and ad hoc post-hoc logic that is easy to get subtly wrong.

`chFnirs` packages the boilerplate so that a full second-level analysis — omnibus plus activation plus gated post-hocs plus heatmaps — fits in a few dozen lines.

1.3 When to use `chFnirs`

Use `chFnirs` when:

- You have a long-format table of per-subject fNIRS estimates (one row per subject × area × condition × chromophore).
- You want to fit the same model at every brain area with BH correction across areas.

- You want a unified, self-documenting output schema that downstream plotting and tabling functions can consume.

You do *not* need `chFnirs` if you are doing single-subject first-level analysis (use your preferred fNIRS toolbox) or if you have a single area and can just call `lmerTest::lmer` / `car::Anova` directly.

2 Installation

`chFnirs` is available from GitHub:

```
# devtools::install_github("ccpluncw/ccpl_R_chFnirs")
```

Key dependencies:

- `lmerTest` — linear mixed models with Satterthwaite df
- `car` — Type III ANOVA
- `emmeans` — estimated marginal means, contrasts, slopes
- `sandwich` — HC0–HC5 robust covariance
- `ggplot2`, `reshape2`, `ggnewscale` — heatmaps and slope plots
- `dplyr`, `magrittr`, `rlang`, `stringr` — data manipulation

Load the package:

```
library(chFnirs)
```

3 The data format `chFnirs` expects

All `chFnirs` analysis functions consume a **long-format data frame** with at minimum:

- A **subject** column (for random effects when using `model = "lmer"`).
- A **type** column naming the chromophore — one of `hbo`, `hbr`, `hbdiff`, `cbsi`, or any label you prefer. Default column name: `type`.
- An **area** column naming the brain area — a channel number, an ROI label, or a region name. You tell every function which column this is via the `area_col` argument.
- One or more **factor columns** for your experimental design.
- An **estimate** column holding the per-cell response amplitude. The column can have any name; pass it as the left-hand side of `formula`.

Rows that don't belong to the analysis should be filtered out before calling `chFnirs`. Internally the package fits a separate model at each (type, area) combination, skipping cells with fewer than `min_rows` observations (default 5).

3.1 Building a synthetic example dataset

The rest of this tutorial uses a small synthetic dataset so every code chunk runs live when the vignette builds. The dataset is shaped like a plausible fNIRS experiment: a between-subjects `Group` factor (Evaluation vs. No-Evaluation), a within-subjects `cond` factor (read vs. speak), 4 ROIs, 4 chromophores, and a continuous between-subjects moderator `zMood` standing in for a trait questionnaire score.

Two effects are planted in the data so that the pipeline produces something interesting:

1. A **Group × cond interaction in ROI1** for the active chromophores (HbO and CBSI): Eval–speak trials are boosted above baseline.

2. A **Group × zMood slope in ROI2** for CBSI: Eval subjects with higher zMood produce larger CBSI responses.

Everything else is noise.

```
set.seed(42)

n_sub    <- 40
subjects <- data.frame(
  ID    = sprintf("S%02d", 1:n_sub),
  Group = rep(c("Eval", "NoEval"), each = n_sub / 2),
  zMood = rnorm(n_sub),
  stringsAsFactors = FALSE
)

rois  <- paste0("ROI", 1:4)
conds <- c("read", "speak")
types <- c("hbo", "hbr", "hbdiff", "cbsi")

design <- expand.grid(
  ID   = subjects$ID,
  ROI  = rois,
  cond = conds,
  type = types,
  stringsAsFactors = FALSE
)
design <- merge(design, subjects, by = "ID")

# --- Plant effects ---
design$estimate <- with(design, {
  # Chromophore baseline: HbO and CBSI are positive, HbR negative, HbDiff near 0
  base <- ifelse(type %in% c("hbo", "cbsi"),  2.0,
          ifelse(type == "hbr",              -1.0,
                                              0.3))
  # Condition main effect: speak > read
  cond_boost <- ifelse(cond == "speak", 0.5, 0)
  # Planted Group x cond interaction in ROI1 (active chromophores only)
  interact   <- ifelse(ROI == "ROI1" & type %in% c("hbo", "cbsi") &
                         Group == "Eval" & cond == "speak", 1.5, 0)
  # Planted Group x zMood slope in ROI2 (CBSI only)
  mood_slope <- ifelse(ROI == "ROI2" & type == "cbsi" &
                         Group == "Eval", 0.8 * zMood, 0)
  noise <- rnorm(length(base), 0, 1.2)
  base + cond_boost + interact + mood_slope + noise
})

# Factors (important for emmeans)
design$Group <- factor(design$Group, levels = c("Eval", "NoEval"))
design$cond  <- factor(design$cond,  levels = c("read", "speak"))

str(design)
```

```
## 'data.frame':    1280 obs. of  7 variables:
##  $ ID      : chr  "S01" "S01" "S01" "S01" ...
##  $ ROI     : chr  "ROI1" "ROI4" "ROI1" "ROI2" ...
```

```
##  $ cond    : Factor w/ 2 levels "read","speak": 1 2 2 2 1 2 1 2 1 2 ...
##  $ type    : chr  "hbo" "cbsi" "cbsi" "cbsi" ...
##  $ Group   : Factor w/ 2 levels "Eval","NoEval": 1 1 1 1 1 1 1 1 1 1 ...
##  $ zMood   : num  1.37 1.37 1.37 1.37 1.37 ...
##  $ estimate: num  2.25 2.07 4.91 2.72 1.45 ...
```

The dataset has 1280 rows — 40 subjects × 4 ROIs × 2 conditions × 4 chromophores — which is a good proxy for a real region-level fNIRS dataset.

4 The categorical pipeline

A typical categorical analysis has three stages:

1. **Omnibus.** Fit a model at every (type, area) combination; run a Type III ANOVA to get an F statistic and p-value for every factor and interaction; BH-correct across areas.
2. **Activation tests.** For each level of a factor, test whether the estimated marginal mean response is different from zero. This answers "did this condition produce measurable activation?".
3. **Gated post-hocs.** For effects that are significant in the omnibus, run pairwise contrasts between factor levels (or between levels of one factor conditional on levels of another).

4.1 Omnibus testing with BH correction

A second-level fNIRS analysis almost always tests the same model at dozens to hundreds of cells (channels × chromophores, or ROIs × chromophores), so the family of tests is large and some form of multiple-comparison correction is required. `chFnirs` uses the **Benjamini–Hochberg (BH) false discovery rate** procedure (Benjamini & Hochberg, 1995). Because a multiway ANOVA tests several effects (main effects, interactions) at every cell, `chFnirs` defaults to BH-correcting within each (chromophore, effect) family — i.e. each effect type (Group, cond, Group:cond, …) is treated as its own family across areas. This treats each effect type as a separate family of hypotheses, reflecting the fact that different effects address different scientific questions and would typically be reported in separate results sections (Bender & Lange, 2001; McFarquhar, 2019). The alternative, pooling all effects into a single correction family, is overly conservative because it penalizes the test of one effect for the number of tests conducted on a conceptually unrelated effect. The default `bh_group_col = c("type", "effect")` in `run_omnibus_by_area_bh()` implements this per-effect-type grouping.

`run_omnibus_by_area_bh()` is the convenience wrapper that combines `run_omnibus_by_area()` and `getBHvalues()` in one call. It fits the specified model at every (type, area) pair and returns a long table with one row per effect × type × area, including both raw and BH-corrected p-values.

```
omnibus_cat <- run_omnibus_by_area_bh(
  data     = design,
  formula  = estimate ~ Group * cond + (1 | ID),
  area_col = "ROI",
  model    = "lmer"
)

head(omnibus_cat, 10)
```

```
##      type area effect  statistic statistic_type df_num   df_den      p_value
## 1    cbsi ROI1   cond 41.9178142              F      1 38.00000 1.275295e-07
## 2    cbsi ROI4   cond  3.1164323              F      1 57.02639 8.286011e-02
## 3    cbsi ROI2   cond  1.0851598              F      1 38.00000 3.041286e-01
## 4    cbsi ROI3   cond  1.2921318              F      1 38.00000 2.627732e-01
## 5  hbdiff ROI1   cond  7.0434308              F      1 50.58437 1.060607e-02
```

```
## 6  hbdiff ROI4   cond  0.5560716              F      1 37.99999 4.604355e-01
## 7  hbdiff ROI2   cond  3.3990377              F      1 38.00000 7.304330e-02
## 8  hbdiff ROI3   cond  3.9736588              F      1 38.00000 5.343230e-02
## 9     hbo ROI1   cond 23.1978621              F      1 37.99954 2.351938e-05
## 10    hbo ROI4   cond  3.4904262              F      1 61.28407 6.650696e-02
##    partial_eta_sq singular         p_bh
## 1              NA    FALSE 5.101178e-07
## 2              NA     TRUE 1.657202e-01
## 3              NA    FALSE 3.041286e-01
## 4              NA    FALSE 3.041286e-01
## 5              NA     TRUE 4.242428e-02
## 6              NA    FALSE 4.604355e-01
## 7              NA    FALSE 9.739107e-02
## 8              NA    FALSE 9.739107e-02
## 9              NA    FALSE 9.407751e-05
## 10             NA     TRUE 6.650696e-02
```

A few things to notice:

- **`model` is a required argument.** There is no default. You must actively choose between `"lmer"` (linear mixed model; use when you have repeated observations per subject) and `"lm"` (ordinary linear model; use for flat, independent-observation data). Omitting it produces an informative error.
- **The output schema is snake_case.** Every downstream function in `chFnirs` expects `type`, `area`, `effect`, `p_value`, `p_bh`, `statistic`, `statistic_type`, `partial_eta_sq`. Do not rename these columns manually — the gating logic in `run_gated_posthoc()` depends on them.
- **BH correction is by type and effect.** The default `bh_group_col = c("type", "effect")` treats every (chromophore, effect) pair as an independent family of tests, which matches the convention used in most published fNIRS analyses. Override it if you want a different grouping.

Let's look at which effects survived BH correction:

```
sig_omnibus <- omnibus_cat[!is.na(omnibus_cat$p_bh) & omnibus_cat$p_bh < 0.05, ]
sig_omnibus[, c("type", "area", "effect", "statistic", "p_value", "p_bh",
                "partial_eta_sq")]
```

```
##      type area     effect statistic      p_value         p_bh partial_eta_sq
## 1    cbsi ROI1       cond 41.917814 1.275295e-07 5.101178e-07             NA
## 5  hbdiff ROI1       cond  7.043431 1.060607e-02 4.242428e-02             NA
## 9     hbo ROI1       cond 23.197862 2.351938e-05 9.407751e-05             NA
## 11    hbo ROI2       cond  6.978549 1.096025e-02 1.461367e-02             NA
## 12    hbo ROI3       cond  8.060565 7.225275e-03 1.445055e-02             NA
## 13    hbr ROI1       cond  8.479983 5.351444e-03 1.070289e-02             NA
## 15    hbr ROI2       cond  8.798630 5.188491e-03 1.070289e-02             NA
## 25    hbo ROI1      Group 14.502594 4.966311e-04 1.986525e-03             NA
## 33   cbsi ROI1 Group:cond 29.870485 3.079365e-06 1.231746e-05             NA
```

We see the planted Group × cond interaction in ROI1 for HbO and CBSI, plus the expected main effect of `cond` on CBSI.

4.2 Activation tests

`run_activation_tests()` asks, for each level of a specified factor, is the estimated marginal mean response different from zero? This is the "is there activation here at all?" question. The test is ungated — it does not depend on the omnibus result — because the question "is there activation" is logically separate from "does activation differ between conditions".

```
activation_cond <- run_activation_tests(
  data      = design,
  formula   = estimate ~ Group * cond + (1 | ID),
  area_col  = "ROI",
  by_factor = "cond",
  model     = "lmer"
)

head(activation_cond)
```

```
##   type area predictor estimate        se       df statistic statistic_type
## 1  hbo ROI1      read 2.116441 0.1983047 75.93497 10.672674              t
## 2  hbo ROI1     speak 3.447272 0.1983047 75.93497 17.383711              t
## 3  hbo ROI4      read 1.852598 0.1961085 76.00000  9.446800              t
## 4  hbo ROI4     speak 2.355199 0.1961085 76.00000 12.009675              t
## 5  hbo ROI2      read 1.965135 0.2100832 76.00000  9.354077              t
## 6  hbo ROI2     speak 2.704576 0.2100832 76.00000 12.873829              t
##        p_value cohens_d
## 1 9.088303e-17 1.712745
## 2 3.441456e-28 2.789728
## 3 1.868938e-14 1.493670
## 4 3.094320e-19 1.898896
## 5 2.808767e-14 1.479009
## 6 8.780745e-21 2.035531
```

Each row reports, for one (type, area, cond) cell: the marginal mean (`estimate`), its standard error, a t-statistic, a raw p-value, and Cohen's d. The `predictor` column is a compact label combining the factor level and, where relevant, its semantic role (here just the level of `cond`).

Because these tests are ungated, they ignore the omnibus. They are also **not automatically BH-corrected** — apply your own correction if needed:

```
activation_cond$p_bh <- p.adjust(activation_cond$p_value, method = "BH")
```

4.3 Gated post-hoc contrasts

When the omnibus reports a significant interaction or main effect, the next question is which specific levels differ. `chFnirs` uses a **gated** testing strategy: pairwise post-hoc contrasts at a given (type, area) cell are only computed when the omnibus test for the named effect at that cell was significant after BH correction. This protects against reporting post-hocs for an effect that is absent across all areas. It does not control error across effects, models, or signals, so users should define their correction families and report them. `run_gated_posthoc()` runs `emmeans`-based pairwise contrasts in exactly this fashion.

To gate on the planted Group × cond interaction, and compare Group levels within each level of cond:

```
posthoc <- run_gated_posthoc(
  omnibus_df      = omnibus_cat,
  effect_name     = "Group:cond",
  data            = design,
  formula         = estimate ~ Group * cond + (1 | ID),
  area_col        = "ROI",
  contrast_factor = "Group",
  by_var          = "cond",
  model           = "lmer"
)
```

```r
head(posthoc, 10)
```

```
##    type area      contrast           predictor   estimate        se       df
## 1   hbo ROI1 Eval - NoEval  read_Eval - NoEval         NA        NA       NA
## 2   hbo ROI1 Eval - NoEval speak_Eval - NoEval         NA        NA       NA
## 3   hbo ROI4 Eval - NoEval  read_Eval - NoEval         NA        NA       NA
## 4   hbo ROI4 Eval - NoEval speak_Eval - NoEval         NA        NA       NA
## 5   hbo ROI2 Eval - NoEval  read_Eval - NoEval         NA        NA       NA
## 6   hbo ROI2 Eval - NoEval speak_Eval - NoEval         NA        NA       NA
## 7   hbo ROI3 Eval - NoEval  read_Eval - NoEval         NA        NA       NA
## 8   hbo ROI3 Eval - NoEval speak_Eval - NoEval         NA        NA       NA
## 9  cbsi ROI1 Eval - NoEval  read_Eval - NoEval -0.5105516 0.3313811 73.81839
## 10 cbsi ROI1 Eval - NoEval speak_Eval - NoEval  1.8202326 0.3313811 73.81839
##    statistic statistic_type        p_bon   cohens_d
## 1         NA              t           NA         NA
## 2         NA              t           NA         NA
## 3         NA              t           NA         NA
## 4         NA              t           NA         NA
## 5         NA              t           NA         NA
## 6         NA              t           NA         NA
## 7         NA              t           NA         NA
## 8         NA              t           NA         NA
## 9  -1.540678              t 1.276711e-01 -0.5353943
## 10  5.492869              t 5.357264e-07  1.9088023
```

The output includes:

- `contrast` — the `emmeans` contrast string (e.g., `"Eval - NoEval"`).
- `predictor` — a combined label identifying the level of `by_var` and the contrast.
- `estimate`, `se`, `df`, `statistic` — the contrast and its test.
- `p_bon` — Bonferroni-adjusted p-value from `emmeans`' internal adjustment across the pairs within each cell.
- `cohens_d` — standardized effect size for the contrast.

Cells where the omnibus was not significant get `NA` in the estimate and p-value columns (if `fill_empty = TRUE`, the default). This keeps the output rectangular, so downstream joins and heatmaps don't need to reason about missing cells.

4.3.1 Why gating is enforced

`run_gated_posthoc()` **errors** if `omnibus_df` doesn't contain the column named in `p_col` (default `"p_bh"`):

```r
# This will error: no p_bh column in `activation_cond`
run_gated_posthoc(
  omnibus_df      = activation_cond,   # <-- wrong table
  effect_name     = "cond",
  # ... rest of args ...
)
# Error: column `p_bh` not found in `omnibus_df`. Pass a table produced
# by run_omnibus_by_area_bh(), or call getBHvalues() on an omnibus table
# first.
```

This is deliberate. Silent auto-BH or silent fallback to raw p-values would produce different effective alpha levels depending on what the user happened to pass in. Making the user pipe through `run_omnibus_by_area_bh()` (or `getBHvalues()`) makes the FDR adjustment visible in the script.

4.4 Effect-size heatmap

`create_effect_size_heatmap()` produces a PDF heatmap of Cohen's d (or partial η^2, or any other effect-size column) across areas, with optional asterisks marking significant cells and per-chromophore splitting.

```
roi_order <- sort_rois_anatomically(unique(activation_cond$area))

create_effect_size_heatmap(
  results_df             = activation_cond,
  area_col               = "ROI",
  es_col                 = "cohens_d",
  p_col                  = "p_bh",          # asterisks where p_bh < threshold
  significance_threshold = 0.05,
  group_col              = "predictor",     # one page per cond level
  area_order             = roi_order,       # caller-supplied x-axis order
  save_dir               = tempdir(),
  file_prefix            = "activation_by_cond",
  split_by_type          = TRUE             # one subdirectory per chromophore
)
```

Two conventions worth calling out:

- `split_by_type = TRUE` (the default on every plotting function in the package) creates `<save_dir>/<type>/` subdirectories and writes one PDF per chromophore. Turn this off only if you have already filtered your data to a single chromophore.
- `area_order` is the heatmap's pluggable-sorter hook. Pass any character vector specifying the desired x-axis order. The two built-in sorters — `sort_rois_anatomically()` for ROI labels and `sort_fnirs_channels_by_name()` for fNIRS channels (which uses a digpts file to compute anatomical position) — are interchangeable; any user-supplied function with the contract `function(names, ...) -> ordered names` works equally well. With `area_order = NULL`, columns appear in the order they occur in `results_df`.

For the CBSI chromophore in our synthetic dataset, the effect-size heatmap that `create_effect_size_heatmap()` produces looks like Figure 1. The planted Group × cond interaction is visible as the large positive d for the speak condition in ROI1, where Eval subjects produce the amplified response.

```
library(ggplot2)

es_df <- activation_cond[activation_cond$type == "cbsi", ]
es_df$sig <- ifelse(!is.na(es_df$p_value) & es_df$p_value < 0.05,
                    "*", "")
es_df$label <- sprintf("%.2f%s", es_df$cohens_d, es_df$sig)

ggplot(es_df, aes(x = area, y = predictor, fill = cohens_d)) +
  geom_tile(color = "white") +
  geom_text(aes(label = label), size = 3.2) +
  scale_fill_gradient2(low = "steelblue", mid = "white",
                       high = "firebrick", midpoint = 0,
                       name = "Cohen's d") +
  labs(x = "ROI", y = "cond level",
       title = "CBSI activation: Cohen's d by cond x ROI") +
  theme_minimal(base_size = 10) +
  theme(panel.grid = element_blank())
```

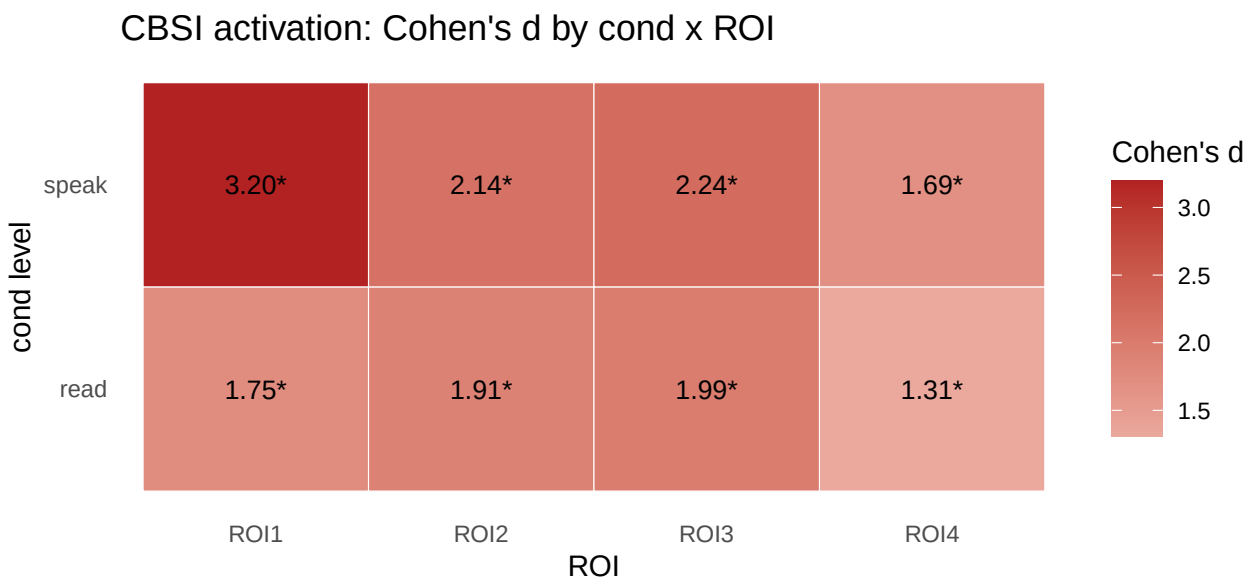


Figure 1: Effect-size heatmap for CBSI activation (Cohen's d for the marginal mean vs. zero within each cond level). Cells flagged with an asterisk are significant at the uncorrected $\alpha = 0.05$ level. The speak condition in ROI1 stands out because of the planted Group × cond interaction in that cell.

5 Adding a continuous moderator

If a continuous variable (a questionnaire score, a hormone concentration, a baseline measure) moderates the categorical effects, fit a separate model that adds the moderator and its interactions. The slope-based pipeline mirrors the categorical pipeline, with two differences:

- `run_slopes_vs_zero()` replaces `run_activation_tests()`. It asks "is the slope of the DV on the moderator different from zero?"
- `run_gated_slopes()` replaces `run_gated_posthoc()`. It asks "do slopes differ between groups / conditions?"

Fit an extended model that includes the moderator, then extract the omnibus as before:

```
h_formula <- estimate ~ Group * cond * zMood + (1 | ID)

omnibus_h <- run_omnibus_by_area_bh(
  data     = design,
  formula  = h_formula,
  area_col = "ROI",
  model    = "lmer"
)

# Keep only effects involving the moderator, for gating the slopes.
omnibus_h_slopes <- omnibus_h[grepl("zMood", omnibus_h$effect), ]
head(omnibus_h_slopes)
```

```
##      type area     effect   statistic statistic_type df_num   df_den   p_value
## 17   cbsi ROI1 cond:zMood 0.159782391              F      1 36.00000 0.6917188
## 18   cbsi ROI4 cond:zMood 0.605025955              F      1 53.31792 0.4401048
## 19   cbsi ROI2 cond:zMood 0.006781892              F      1 36.00000 0.9348228
## 20   cbsi ROI3 cond:zMood 1.122158075              F      1 36.00000 0.2965103
## 21 hbdiff ROI1 cond:zMood 0.052521853              F      1 47.88166 0.8197079
```

```
## 22 hbdiff ROI4 cond:zMood 0.269558475              F      1 36.00000 0.6068055
##    partial_eta_sq singular      p_bh
## 17             NA    FALSE 0.9222917
## 18             NA     TRUE 0.8802096
## 19             NA    FALSE 0.9348228
## 20             NA    FALSE 0.8802096
## 21             NA     TRUE 0.8291828
## 22             NA    FALSE 0.8291828
```

Filtering the omnibus table to only moderator-containing effects is done script-side on purpose: it keeps the two-model structure (categorical model → categorical post-hocs, moderator model → slope post-hocs) visible to the analyst.

5.1 Slopes vs. zero

```
slopes_by_group <- run_slopes_vs_zero(
  data      = design,
  formula   = h_formula,
  area_col  = "ROI",
  trend_var = "zMood",
  by_factor = "Group",
  model     = "lmer"
)

head(slopes_by_group, 8)
```

```
##   type area          predictor    estimate        se df  statistic
## 1  hbo ROI1   Eval_zMood_Slope  0.02494035 0.1581544 36  0.1576962
## 2  hbo ROI1 NoEval_zMood_Slope  0.23232683 0.1870836 36  1.2418345
## 3  hbo ROI4   Eval_zMood_Slope -0.15424101 0.1523162 36 -1.0126368
## 4  hbo ROI4 NoEval_zMood_Slope  0.25759191 0.1801774 36  1.4296568
## 5  hbo ROI2   Eval_zMood_Slope  0.19287670 0.1608776 36  1.1989035
## 6  hbo ROI2 NoEval_zMood_Slope -0.41969731 0.1903048 36 -2.2053952
## 7  hbo ROI3   Eval_zMood_Slope -0.11470039 0.1461354 36 -0.7848913
## 8  hbo ROI3 NoEval_zMood_Slope -0.29415911 0.1728660 36 -1.7016594
##   statistic_type    p_value    cohens_d
## 1              t 0.87557741  0.02064936
## 2              t 0.22232758  0.19235490
## 3              t 0.31799163 -0.12514690
## 4              t 0.16143705  0.20900297
## 5              t 0.23839787  0.14816671
## 6              t 0.03389788 -0.32240893
## 7              t 0.43765225 -0.11054698
## 8              t 0.09744194 -0.28350733
```

One row per (type, area, Group-level). The `predictor` column is labelled `"<level>_<trend_var>_Slope"` (e.g., `Eval_zMood_Slope`), so if you need the bare group level for a downstream plot, strip that suffix. `estimate` is the slope value, `cohens_d` is Cohen's d for the slope-vs-zero test, and `p_value` is the raw p-value (apply your own BH correction if you want).

5.2 Gated slopes: do groups have different slopes?

`run_gated_slopes()` uses `emmeans::emtrends` to test whether slopes differ between levels of one or more categorical factors, gated on the omnibus. It auto-decomposes the interaction: for a two-way Group×zMood effect, it tests Eval-vs-NoEval slopes; for a three-way Group×cond×zMood effect, it further splits the comparison by cond.

```
gated_slopes <- run_gated_slopes(
  omnibus_df          = omnibus_h_slopes,
  data                = design,
  formula             = h_formula,
  area_col            = "ROI",
  trend_var           = "zMood",
  categorical_factors = c("Group", "cond"),
  model               = "lmer"
)

# Show only the cells that were actually tested (not NA-filled).
tested <- gated_slopes[!is.na(gated_slopes$p_bon), ]
head(tested)
```

```
##    type area omnibus_effect posthoc_type contrast          predictor estimate
## 49 cbsi ROI2          zMood      Overall     <NA> zMood_Slope_vsZero 0.384728
##           se df statistic statistic_type       p_bon  cohens_d
## 49 0.1288983 36   2.98474              t 0.005075635 0.3506025
```

The `omnibus_effect` column tells you which interaction in the omnibus drove this row, and `posthoc_type` tells you what kind of decomposition was applied (single-factor pairwise, a two-way comparison, or a three-way conditional-slice).

5.3 Visualizing slopes

Two plotting helpers, both returning `invisible(result)` and writing per-chromophore PDFs:

- `plot_simple_slopes()` — for continuous-×-continuous interactions; plots one line per $±$1 SD level of a second continuous moderator.
- `plot_slope_lines()` — for categorical-×-continuous interactions; plots one line per group (or group × condition cell) across the moderator range.

For our Group×zMood effect, `plot_slope_lines()` is the right tool:

```
plot_slope_lines(
  results_df = slopes_by_group,
  data       = design,
  formula    = h_formula,
  area_col   = "ROI",
  trend_var  = "zMood",
  by_factor  = "Group",
  model      = "lmer",
  p_col      = "p_value",
  pdf_prefix = "zMood_byGroup",
  output_dir = tempdir()
)
```

The PDF filename convention is `<pdf_prefix>_<type>_SlopeLines.pdf`; `plot_simple_slopes()` uses `<pdf_prefix>_<type>_SimpleSlopes.pdf`. Do not append `_SlopeLines` or `_SimpleSlopes` to `pdf_prefix`

yourself — the function adds it.

The slope-lines figure `plot_slope_lines()` produces for this dataset shows one line per Group within each ROI (Figure 2). The Group × zMood interaction planted in ROI2 for CBSI is visible as a clear positive slope for Eval subjects against a flat slope for NoEval; the other ROIs show no systematic Group-specific moderation, as expected.

```
library(ggplot2)

# Subject-level means across condition for plotting the raw points
subj_means <- aggregate(estimate ~ ID + ROI + Group + zMood,
                        data = design[design$type == "cbsi", ],
                        FUN = mean)

# Model-predicted slopes by Group x ROI
pred_grid <- expand.grid(
  zMood = seq(min(design$zMood), max(design$zMood), length.out = 40),
  Group = levels(design$Group),
  ROI   = levels(factor(design$ROI)),
  cond  = levels(design$cond),
  stringsAsFactors = FALSE
)

# Fit one lmer per ROI (cbsi only), predict at the grid, aggregate over cond
pred_rows <- do.call(rbind, lapply(unique(design$ROI), function(r) {
  dsub <- design[design$type == "cbsi" & design$ROI == r, ]
  fit <- lmerTest::lmer(estimate ~ Group * cond * zMood + (1 | ID),
                        data = dsub)
  gsub <- pred_grid[pred_grid$ROI == r, ]
  gsub$estimate <- predict(fit, newdata = gsub, re.form = NA)
  # SE via bootstrap would be heavy; use emmeans for clean CIs
  em <- emmeans::emmeans(fit, ~ Group | zMood,
                         at = list(zMood = unique(gsub$zMood)))
  em_df <- as.data.frame(em)
  em_df$ROI <- r
  em_df
}))

names(pred_rows)[names(pred_rows) == "emmean"] <- "estimate"

ggplot(pred_rows,
       aes(x = zMood, y = estimate, color = Group, fill = Group)) +
  geom_ribbon(aes(ymin = lower.CL, ymax = upper.CL),
              alpha = 0.15, color = NA) +
  geom_line(linewidth = 0.9) +
  geom_point(data = subj_means,
             aes(x = zMood, y = estimate, color = Group),
             alpha = 0.5, size = 1.2, inherit.aes = FALSE) +
  facet_wrap(~ ROI, nrow = 2) +
  scale_color_manual(values = c(Eval = "firebrick", NoEval = "steelblue")) +
  scale_fill_manual( values = c(Eval = "firebrick", NoEval = "steelblue")) +
  labs(x = "zMood", y = "CBSI estimate (model-predicted)",
       title = "CBSI activation ~ Group x zMood, by ROI") +
  theme_minimal(base_size = 10)
```

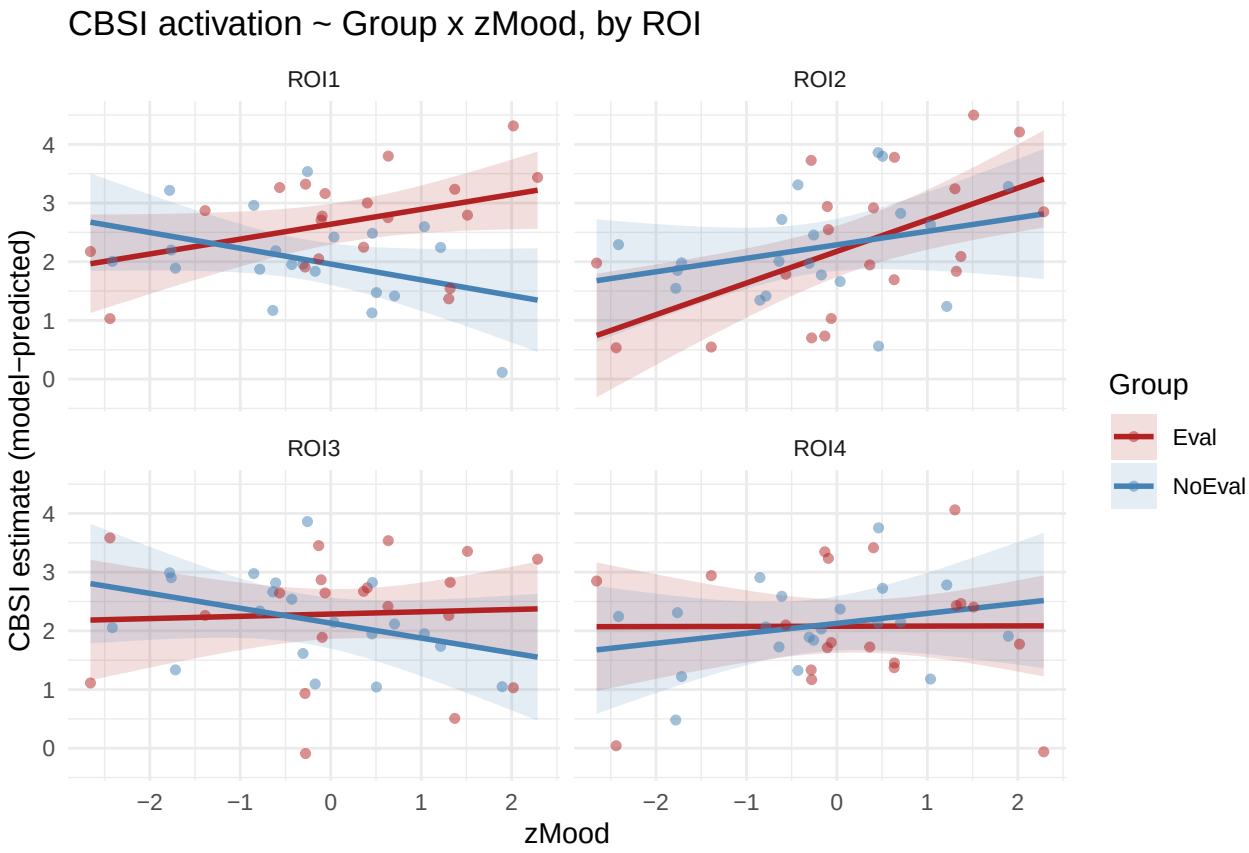


Figure 2: Slope-lines visualization of the Group x zMood interaction for CBSI, faceted by ROI. Lines are model-predicted activation at each level of zMood; shaded ribbons are 95% confidence intervals; points are observed subject means collapsed across condition. ROI2 shows the planted interaction: Eval subjects with higher zMood produce larger CBSI activation, while NoEval subjects show no moderation.

6 Writing results by chromophore

`write_results_by_type()` is a drop-in replacement for `write.csv()` that splits a multi-chromophore results table into per-type subdirectories:

```
# Top-level GroupCond directory layout:
#   tempdir()/hbo/omnibus_GroupCond.csv
#   tempdir()/hbr/omnibus_GroupCond.csv
#   tempdir()/hbdiff/omnibus_GroupCond.csv
#   tempdir()/cbsi/omnibus_GroupCond.csv
write_results_by_type(
  df         = omnibus_cat,
  filename   = "omnibus_GroupCond.csv",
  output_dir = tempdir()
)

# Set split_by_type = FALSE to write one combined CSV instead.
write_results_by_type(
  df            = omnibus_cat,
  filename      = "omnibus_GroupCond_all.csv",
  output_dir    = tempdir(),
  split_by_type = FALSE
)
```

Using this function consistently across your analysis script keeps the output directory structure predictable — every CSV and every plotting call end up in parallel `<type>/` subdirectories.

7 Design principles to know

Five recurring conventions underlie every `chFnirs` analysis function. Knowing them up front will save hours of debugging.

7.1 `model` is required

Every function that fits a model has `model` as a required argument with no default. The choice between `"lmer"` and `"lm"` is a statistical decision, not a convenience setting:

- `"lmer"` fits a linear mixed model via `lmerTest::lmer`. Use when subjects contribute multiple observations and you need random intercepts or slopes.
- `"lm"` fits an ordinary linear model via `stats::lm`. Use for flat, independent-observation data (e.g., already-collapsed subject means).

Passing `model` is required at every call site. This is deliberate — silently defaulting either way would mis-serve half the use cases.

7.2 `vcov` has a conditional default

`vcov` controls the covariance estimator for hypothesis tests. Accepted values: `"classical"`, `"HC0"`, `"HC1"`, `"HC2"`, `"HC3"`, `"HC4"`, `"HC5"`. The default is `NULL`, which resolves to:

- `"classical"` when `model = "lmer"` (the mixed model's native covariance; no robust adjustment).
- `"HC3"` when `model = "lm"` (Long & Ervin's small-sample-friendly heteroscedasticity-consistent covariance).

Combining `model = "lmer"` with any `"HC*"` value raises an explicit error — HC0–HC5 are defined only for OLS residuals and have no analogue for mixed models.

7.3 Snake-case output schema

Every analysis function returns a data frame with snake-case columns: `type`, `area`, `effect`, `predictor`, `contrast`, `estimate`, `se`, `df`, `statistic`, `statistic_type`, `p_value`, `p_bh`, `p_bon`, `partial_eta_sq`, `cohens_d`, `omnibus_effect`, `posthoc_type`. Every downstream function in the package — heatmap generators, slope plots, effect-size utilities — expects these exact column names.

Renaming columns manually will break the pipeline. If you have a legacy analysis that uses old CamelCase column names (`Type`, `Area`, `Predictor`, `pVal`, `pBH`, `Effect`, `F value`), migrate to the new schema or isolate the legacy code in its own script.

7.4 Gated functions error on missing gating column

Any function that "gates" on an omnibus result (`run_gated_posthoc()`, `run_gated_slopes()`, `plot_simple_slopes()`) errors if the `p_col` you asked it to gate on (default `"p_bh"`) is not present in `omnibus_df`. There is no silent fallback to raw p-values. If you want to gate on a different column (raw `p_value`, Bonferroni `p_bon`, etc.), pass `p_col = "p_value"` explicitly.

This is the same philosophy as requiring `model`: make statistical decisions visible at the call site.

7.5 Effect sizes are always computed

Every analysis function always returns an effect-size column (`partial_eta_sq` for omnibus, `cohens_d` for activation / slopes / post-hocs), even when the underlying computation is undefined. In those cases the column is `NA_real_` rather than absent. This keeps the output schema rectangular so that downstream code doesn't need to branch on whether an effect size is available.

An `lmer` omnibus, for example, always has `partial_eta_sq = NA` because `lmerTest` doesn't produce a sum-of-squares column. An `lm` omnibus produces real values. The column is present either way.

8 Common pitfalls

"Argument `model` is required." You forgot `model = "lmer"` or `model = "lm"`. Every analysis function requires it.

"`vcov = "HC3"` is not valid when `model = "lmer"`." HC0–HC5 only work with `lm`. For mixed models, use `vcov = "classical"` (or leave `vcov = NULL` to get the classical default automatically).

"column `p_bh` not found in `omnibus_df`." You passed a table that never went through BH correction. Either (a) build the omnibus table via `run_omnibus_by_area_bh()` instead of `run_omnibus_by_area()`, (b) pipe the raw omnibus through `getBHvalues()` first, or (c) pass `p_col = "p_value"` to gate on the raw omnibus p-value instead.

Heatmap columns in the wrong order. Heatmaps render columns in the order specified by `area_order`, which defaults to `unique(results_df[[area_col]])` (i.e., order of occurrence). For ROI data, pre-compute `area_order = sort_rois_anatomically(unique(...))` and pass it. For channel data, use `sort_fnirs_channels_by_name(...)` with a digpts file. Either function returns a character vector satisfying the heatmap's pluggable-sorter contract.

`pdf_prefix` ends in `_SlopeLines` or `_SimpleSlopes`. The plotting functions append those suffixes themselves, so a prefix of `"zMood_byGroup_SlopeLines"` becomes `zMood_byGroup_SlopeLines_cbsi_SlopeLines.pdf`. Drop the suffix from your prefix string.

Factors stored as character. `emmeans` needs factor variables to identify levels. If your script builds the data frame with `stringsAsFactors = FALSE`, wrap categorical columns in `factor()` with explicit levels before calling any `chFnirs` function, so the reference level and contrast order are under your control.

9 Summary

A complete second-level analysis in `chFnirs` follows the same template for every experiment:

Stage	Function	Purpose
Omnibus	`run_omnibus_by_area_bh()`	Area-wise Type III ANOVA with BH across areas
Activation	`run_activation_tests()`	Marginal-mean activation vs. zero
Slopes	`run_slopes_vs_zero()`	Slope of DV on continuous moderator vs. zero
Gated posthoc	`run_gated_posthoc()`	Categorical pairwise contrasts, gated on omnibus
Gated slopes	`run_gated_slopes()`	Slope contrasts between groups/conditions, gated on omnibus
Visualization	`create_effect_size_heatmap()`, `create_pvalue_heatmap()`, `plot_slope_lines()`, `plot_simple_slopes()`	PDF heatmaps and slope plots
I/O	`write_results_by_type()`	Per-chromophore CSVs in `<type>/` subdirectories

Every function takes the same `data`, `formula`, `area_col`, `type_col`, `model`, and `vcov` arguments in the same way, so once you know the pattern for one function you know it for all of them.

For detailed argument documentation, see the **chFnirs Reference Manual** vignette (`vignette("chFnirs_reference", package = "chFnirs")`) or the per-function help pages.

10 Acknowledgments

This document was drafted with assistance from Claude (Anthropic). All technical content reflects the authors' design decisions and domain expertise.

11 References

Bender, R., & Lange, S. (2001). Adjusting for multiple testing — when and how? *Journal of Clinical Epidemiology, 54*(4), 343–349.

Benjamini, Y., & Hochberg, Y. (1995). Controlling the false discovery rate: A practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B, 57*(1), 289–300.

Cui, X., Bray, S., & Reiss, A. L. (2010). Functional near infrared spectroscopy (NIRS) signal improvement based on negative correlation between oxygenated and deoxygenated hemoglobin dynamics. *NeuroImage, 49*(4), 3039–3046.

Long, J. S., & Ervin, L. H. (2000). Using heteroscedasticity consistent standard errors in the linear regression model. *The American Statistician, 54*(3), 217–224.

McFarquhar, M. (2019). Modeling group-level repeated measurements of neuroimaging data using the univariate general linear model. *Frontiers in Neuroscience, 13*, 352.

Santosa, H., Zhai, X., Fishburn, F., & Huppert, T. (2018). The NIRS Brain AnalyzIR Toolbox. *Algorithms, 11*(5), 73.